

Source code for nltk.text

```
# Natural Language Toolkit: Texts
#
# Copyright (C) 2001-2017 NLTK Project
# Author: Steven Bird <stevenbird1@gmail.com>
#         Edward Loper <edloper@gmail.com>
# URL: <http://nltk.org/>
# For license information, see LICENSE.TXT

"""
This module brings together a variety of NLTK functionality for
text analysis, and provides simple, interactive interfaces.
Functionality includes: concordancing, collocation discovery,
regular expression search over tokenized strings, and
distributional similarity.
"""

from __future__ import print_function, division, unicode_literals

from math import log
from collections import defaultdict, Counter
from functools import reduce
from itertools import islice
import re

from six import text_type

from nltk.probability import FreqDist, LidstoneProbDist
from nltk.probability import ConditionalFreqDist as CFD
from nltk.util import tokenwrap, LazyConcatenation
from nltk.metrics import f_measure, BigramAssocMeasures
from nltk.collocations import BigramCollocationFinder
from nltk.compat import python_2_unicode_compatible

class ContextIndex(object):
    """
    A bidirectional index between words and their 'contexts' in a text.
    The context of a word is usually defined to be the words that occur
    in a fixed window around the word; but other definitions may also
    be used by providing a custom context function.
    """
    @staticmethod
    def _default_context(tokens, i):
        """One left token and one right token, normalized to lowercase"""
        left = (tokens[i-1].lower() if i != 0 else '*START*')
        right = (tokens[i+1].lower() if i != len(tokens) - 1 else '*END*')
        return (left, right)

    def __init__(self, tokens, context_func=None, filter=None, key=lambda x:x):
        self._key = key
        self._tokens = tokens
        if context_func:
            self._context_func = context_func
        else:
            self._context_func = self._default_context
        if filter:
            tokens = [t for t in tokens if filter(t)]
        self._word_to_contexts = CFD((self._key(w), self._context_func(tokens, i))
                                     for i, w in enumerate(tokens))
        self._context_to_words = CFD((self._context_func(tokens, i), self._key(w))
                                     for i, w in enumerate(tokens))

    def tokens(self):
        """
```

TABLE OF CONTENTS

NLTK News
Installing NLTK
Installing NLTK Data
Contribute to NLTK
FAQ
Wiki
API
HOWTO

SEARCH

Go

[\[docs\]](#)

[\[docs\]](#)

```

:rtype: List(str)
:return: The document that this context index was
        created from.
"""
return self._tokens

```

```

def word_similarity_dict(self, word): \[docs\]
    """
    Return a dictionary mapping from words to 'similarity scores,'
    indicating how often these two words occur in the same
    context.
    """
    word = self._key(word)
    word_contexts = set(self._word_to_contexts[word])

    scores = {}
    for w, w_contexts in self._word_to_contexts.items():
        scores[w] = f_measure(word_contexts, set(w_contexts))

    return scores

```

```

def similar_words(self, word, n=20): \[docs\]
    scores = defaultdict(int)
    for c in self._word_to_contexts[self._key(word)]:
        for w in self._context_to_words[c]:
            if w != word:
                scores[w] += self._context_to_words[c][word] * self._context_to_words[c][w]
    return sorted(scores, key=scores.get, reverse=True)[:n]

```

```

def common_contexts(self, words, fail_on_unknown=False): \[docs\]
    """
    Find contexts where the specified words can all appear; and
    return a frequency distribution mapping each context to the
    number of times that context was used.

    :param words: The words used to seed the similarity search
    :type words: str
    :param fail_on_unknown: If true, then raise a value error if
        any of the given words do not occur at all in the index.
    """
    words = [self._key(w) for w in words]
    contexts = [set(self._word_to_contexts[w]) for w in words]
    empty = [words[i] for i in range(len(words)) if not contexts[i]]
    common = reduce(set.intersection, contexts)
    if empty and fail_on_unknown:
        raise ValueError("The following word(s) were not found:",
                          " ".join(words))
    elif not common:
        # nothing in common -- just return an empty freqdist.
        return FreqDist()
    else:
        fd = FreqDist(c for w in words
                      for c in self._word_to_contexts[w]
                      if c in common)
        return fd

```

```

@python_2_unicode_compatible
class ConcordanceIndex(object): \[docs\]
    """
    An index that can be used to look up the offset locations at which
    a given word occurs in a document.
    """
    def __init__(self, tokens, key=lambda x:x):
        """
        Construct a new concordance index.

        :param tokens: The document (list of tokens) that this

```

```

        concordance index was created from. This list can be used
        to access the context of a given word occurrence.
:param key: A function that maps each token to a normalized
        version that will be used as a key in the index. E.g., if
        you use ``key=lambda s:s.lower()`, then the index will be
        case-insensitive.
"""

self._tokens = tokens
"""The document (list of tokens) that this concordance index
    was created from."""

self._key = key
"""Function mapping each token to an index key (or None)."""

self._offsets = defaultdict(list)
"""Dictionary mapping words (or keys) to lists of offset
    indices."""

# Initialize the index (self._offsets)
for index, word in enumerate(tokens):
    word = self._key(word)
    self._offsets[word].append(index)

def tokens(self):
    """
    :rtype: list(str)
    :return: The document that this concordance index was
        created from.
    """
    return self._tokens

def offsets(self, word):
    """
    :rtype: list(int)
    :return: A list of the offset positions at which the given
        word occurs. If a key function was specified for the
        index, then given word's key will be looked up.
    """
    word = self._key(word)
    return self._offsets[word]

def __repr__(self):
    return '<ConcordanceIndex for %d tokens (%d types)>' % (
        len(self._tokens), len(self._offsets))

def print_concordance(self, word, width=75, lines=25):
    """
    Print a concordance for ``word`` with the specified context window.

    :param word: The target word
    :type word: str
    :param width: The width of each line, in characters (default=80)
    :type width: int
    :param lines: The number of lines to display (default=25)
    :type lines: int
    """
    half_width = (width - len(word) - 2) // 2
    context = width // 4 # approx number of words of context

    offsets = self.offsets(word)
    if offsets:
        lines = min(lines, len(offsets))
        print("Displaying %s of %s matches:" % (lines, len(offsets)))
        for i in offsets:
            if lines <= 0:
                break
            left = (' ' * half_width +
                    ' '.join(self._tokens[i-context:i]))
            right = ' '.join(self._tokens[i+1:i+context])

```

[\[docs\]](#)[\[docs\]](#)[\[docs\]](#)

```

        left = left[-half_width:]
        right = right[:half_width]
        print(left, self._tokens[i], right)
        lines -= 1
    else:
        print("No matches")

```

class `TokenSearcher(object):`

[\[docs\]](#)

```

"""
A class that makes it easier to use regular expressions to search
over tokenized strings. The tokenized string is converted to a
string where tokens are marked with angle brackets -- e.g.,
``<the><window><is><still><open>``. The regular expression
passed to the ``findall()`` method is modified to treat angle
brackets as non-capturing parentheses, in addition to matching the
token boundaries; and to have ``'.'`` not match the angle brackets.
"""

def __init__(self, tokens):
    self._raw = ''.join('<'+w+'>' for w in tokens)

def findall(self, regexp):
    """
    Find instances of the regular expression in the text.
    The text is a list of tokens, and a regexp pattern to match
    a single token must be surrounded by angle brackets. E.g.

    >>> from nltk.text import TokenSearcher
    >>> print('hack'); from nltk.book import text1, text5, text9
    hack...
    >>> text5.findall("<.*><.*><bro>")
    you rule bro; telling you bro; u twizted bro
    >>> text1.findall("<a><.*><man>")
    monied; nervous; dangerous; white; white; white; pious; queer; good;
    mature; white; Cape; great; wise; wise; butterless; white; fiendish;
    pale; furious; better; certain; complete; dismasted; younger; brave;
    brave; brave; brave
    >>> text9.findall("<th.*>{3,}")
    thread through those; the thought that; that the thing; the thing
    that; that that thing; through these than through; them that the;
    through the thick; them that they; thought that the

    :param regexp: A regular expression
    :type regexp: str
    """
    # preprocess the regular expression
    regexp = re.sub(r'\s', '', regexp)
    regexp = re.sub(r'<', '(<?', regexp)
    regexp = re.sub(r'>', ')>', regexp)
    regexp = re.sub(r'(?<!\)\)\. ', '[^>]', regexp)

    # perform the search
    hits = re.findall(regexp, self._raw)

    # Sanity check
    for h in hits:
        if not h.startswith('<') and h.endswith('>'):
            raise ValueError('Bad regexp for TokenSearcher.findall')

    # postprocess the output
    hits = [h[1:-1].split('><') for h in hits]
    return hits

```

[\[docs\]](#)

@python_2_unicode_compatible

class `Text(object):`

[\[docs\]](#)

```

"""
A wrapper around a sequence of simple (string) tokens, which is
intended to support initial exploration of texts (via the
interactive console). Its methods perform a variety of analyses

```

on the text's contexts (e.g., counting, concordancing, collocation discovery), and display the results. If you wish to write a program which makes use of these analyses, then you should bypass the `Text` class, and use the appropriate analysis function or class directly instead.

A `Text` is typically initialized from a given document or corpus. E.g.:

```
>>> import nltk.corpus
>>> from nltk.text import Text
>>> moby = Text(nltk.corpus.gutenberg.words('melville-moby_dick.txt'))

"""
# This defeats Lazy Loading, but makes things faster. This
# *shouldn't* be necessary because the corpus view *should* be
# doing intelligent caching, but without this it's running slow.
# Look into whether the caching is working correctly.
_COPY_TOKENS = True

def __init__(self, tokens, name=None):
    """
    Create a Text object.

    :param tokens: The source text.
    :type tokens: sequence of str
    """
    if self._COPY_TOKENS:
        tokens = list(tokens)
    self.tokens = tokens

    if name:
        self.name = name
    elif ']' in tokens[:20]:
        end = tokens[:20].index(']')
        self.name = " ".join(text_type(tok) for tok in tokens[1:end])
    else:
        self.name = " ".join(text_type(tok) for tok in tokens[:8]) + "..."

#####
# Support item & slice access
#####

def __getitem__(self, i):
    if isinstance(i, slice):
        return self.tokens[i.start:i.stop]
    else:
        return self.tokens[i]

def __len__(self):
    return len(self.tokens)

#####
# Interactive console methods
#####

def concordance(self, word, width=79, lines=25):
    """
    Print a concordance for ``word`` with the specified context window.
    Word matching is not case-sensitive.
    :seealso: ``ConcordanceIndex``
    """
    if '_concordance_index' not in self.__dict__:
        #print("Building index...")
        self._concordance_index = ConcordanceIndex(self.tokens,
                                                    key=lambda s:s.lower())

    self._concordance_index.print_concordance(word, width, lines)
```

[\[docs\]](#)

```

def collocations(self, num=20, window_size=2): \[docs\]
    """
    Print collocations derived from the text, ignoring stopwords.

    :seealso: find_collocations
    :param num: The maximum number of collocations to print.
    :type num: int
    :param window_size: The number of tokens spanned by a collocation (default=2)
    :type window_size: int
    """
    if not ('_collocations' in self.__dict__ and self._num == num and self._window_size == window_size):
        self._num = num
        self._window_size = window_size

        #print("Building collocations list")
        from nltk.corpus import stopwords
        ignored_words = stopwords.words('english')
        finder = BigramCollocationFinder.from_words(self.tokens, window_size)
        finder.apply_freq_filter(2)
        finder.apply_word_filter(lambda w: len(w) < 3 or w.lower() in ignored_words)
        bigram_measures = BigramAssocMeasures()
        self._collocations = finder.nbest(bigram_measures.likelihood_ratio, num)
        colloc_strings = [w1+' '+w2 for w1, w2 in self._collocations]
        print(tokenwrap(colloc_strings, separator="; "))

def count(self, word): \[docs\]
    """
    Count the number of times this word appears in the text.
    """
    return self.tokens.count(word)

def index(self, word): \[docs\]
    """
    Find the index of the first occurrence of the word in the text.
    """
    return self.tokens.index(word)

def readability(self, method): \[docs\]
    # code from nltk_contrib.readability
    raise NotImplementedError

def similar(self, word, num=20): \[docs\]
    """
    Distributional similarity: find other words which appear in the
    same contexts as the specified word; list most similar words first.

    :param word: The word used to seed the similarity search
    :type word: str
    :param num: The number of words to generate (default=20)
    :type num: int
    :seealso: ContextIndex.similar_words()
    """
    if '_word_context_index' not in self.__dict__:
        #print('Building word-context index...')
        self._word_context_index = ContextIndex(self.tokens,
                                                filter=lambda x:x.isalpha(),
                                                key=lambda s:s.lower())

#     words = self._word_context_index.similar_words(word, num)

word = word.lower()
wci = self._word_context_index._word_to_contexts
if word in wci.conditions():
    contexts = set(wci[word])
    fd = Counter(w for w in wci.conditions() for c in wci[w]
                 if c in contexts and not w == word)
    words = [w for w, _ in fd.most_common(num)]

```

```

    print(tokenwrap(words))
else:
    print("No matches")

```

```

def common_contexts(self, words, num=20): \[docs\]
    """
    Find contexts where the specified words appear; list
    most frequent common contexts first.

    :param word: The word used to seed the similarity search
    :type word: str
    :param num: The number of words to generate (default=20)
    :type num: int
    :seealso: ContextIndex.common_contexts()
    """
    if '_word_context_index' not in self.__dict__:
        #print('Building word-context index...')
        self._word_context_index = ContextIndex(self.tokens,
                                                key=lambda s:s.lower())

    try:
        fd = self._word_context_index.common_contexts(words, True)
        if not fd:
            print("No common contexts were found")
        else:
            ranked_contexts = [w for w, _ in fd.most_common(num)]
            print(tokenwrap(w1+"_"+w2 for w1,w2 in ranked_contexts))

    except ValueError as e:
        print(e)

```

```

def dispersion_plot(self, words): \[docs\]
    """
    Produce a plot showing the distribution of the words through the text.
    Requires pylab to be installed.

    :param words: The words to be plotted
    :type words: List(str)
    :seealso: nltk.draw.dispersion_plot()
    """
    from nltk.draw import dispersion_plot
    dispersion_plot(self, words)

```

```

def generate(self, words): \[docs\]
    """
    Issues a reminder to users following the book online
    """
    import warnings
    warnings.warn('The generate() method is no longer available.', DeprecationWarning)

```

```

def plot(self, *args): \[docs\]
    """
    See documentation for FreqDist.plot()
    :seealso: nltk.prob.FreqDist.plot()
    """
    self.vocab().plot(*args)

```

```

def vocab(self): \[docs\]
    """
    :seealso: nltk.prob.FreqDist
    """
    if "_vocab" not in self.__dict__:
        #print("Building vocabulary index...")
        self._vocab = FreqDist(self)
    return self._vocab

```

```

def findall(self, regexp): [docs]
    """
    Find instances of the regular expression in the text.
    The text is a list of tokens, and a regexp pattern to match
    a single token must be surrounded by angle brackets. E.g.

    >>> print('hack'); from nltk.book import text1, text5, text9
    hack...
    >>> text5.findall("<.*><.*><bro>")
    you rule bro; telling you bro; u twizted bro
    >>> text1.findall("<a><.*><man>")
    monted; nervous; dangerous; white; white; white; pious; queer; good;
    mature; white; Cape; great; wise; wise; butterless; white; fiendish;
    pale; furious; better; certain; complete; dismasted; younger; brave;
    brave; brave; brave
    >>> text9.findall("<th.*>{3,}")
    thread through those; the thought that; that the thing; the thing
    that; that that thing; through these than through; them that the;
    through the thick; them that they; thought that the

    :param regexp: A regular expression
    :type regexp: str
    """

    if "_token_searcher" not in self.__dict__:
        self._token_searcher = TokenSearcher(self)

    hits = self._token_searcher.findall(regexp)
    hits = [' '.join(h) for h in hits]
    print(tokenwrap(hits, "; "))

#####
# Helper Methods
#####

_CONTEXT_RE = re.compile('\w+|[\.\!\?]\?')
def _context(self, tokens, i):
    """
    One left & one right token, both case-normalized. Skip over
    non-sentence-final punctuation. Used by the ``ContextIndex``
    that is created for ``similar()`` and ``common_contexts()``.
    """
    # Left context
    j = i-1
    while j>=0 and not self._CONTEXT_RE.match(tokens[j]):
        j -= 1
    left = (tokens[j] if j != 0 else '*START*')

    # Right context
    j = i+1
    while j<len(tokens) and not self._CONTEXT_RE.match(tokens[j]):
        j += 1
    right = (tokens[j] if j != len(tokens) else '*END*')

    return (left, right)

#####
# String Display
#####

def __str__(self):
    return '<Text: %s>' % self.name

def __repr__(self):
    return '<Text: %s>' % self.name

```


Prototype only; this approach will be slow to load

class `TextCollection`(`Text`):

[\[docs\]](#)

"""A collection of texts, which can be loaded with list of texts, or with a corpus consisting of one or more texts, and which supports counting, concordancing, collocation discovery, etc. Initialize a TextCollection as follows:

```
>>> import nltk.corpus
>>> from nltk.text import TextCollection
>>> print('hack'); from nltk.book import text1, text2, text3
hack...
>>> gutenbergs = TextCollection(nltk.corpus.gutenberg)
>>> mytexts = TextCollection([text1, text2, text3])
```

Iterating over a TextCollection produces all the tokens of all the texts in order.

"""

def `__init__`(`self`, `source`):

if hasattr(source, 'words'): # bridge to the text corpus reader
`source = [source.words(f) for f in source.fileids()]`

`self._texts = source`
`Text.__init__(self, LazyConcatenation(source))`
`self._idf_cache = {}`

def `tf`(`self`, `term`, `text`):

[\[docs\]](#)

""" The frequency of the term in text. """
`return text.count(term) / len(text)`

def `idf`(`self`, `term`):

[\[docs\]](#)

""" The number of texts in the corpus divided by the number of texts that the term appears in. If a term does not appear in the corpus, 0.0 is returned. """
idf values are cached for performance.
`idf = self._idf_cache.get(term)`
if idf is None:
`matches = len([True for text in self._texts if term in text])`
FIXME Should this raise some kind of error instead?
`idf = (log(len(self._texts) / matches) if matches else 0.0)`
`self._idf_cache[term] = idf`
`return idf`

def `tf_idf`(`self`, `term`, `text`):

[\[docs\]](#)

`return self.tf(term, text) * self.idf(term)`

def `demo`():

```
from nltk.corpus import brown
text = Text(brown.words(categories='news'))
print(text)
print()
print("Concordance:")
text.concordance('news')
print()
print("Distributionally similar words:")
text.similar('news')
print()
print("Collocations:")
text.collocations()
print()
#print("Automatically generated text:")
#text.generate()
#print()
print("Dispersion plot:")
text.dispersion_plot(['news', 'report', 'said', 'announced'])
print()
print("Vocabulary plot:")
text.plot(50)
```

```
print()
print("Indexing:")
print("text[3]:", text[3])
print("text[3:5]:", text[3:5])
print("text.vocab()['news']:", text.vocab()['news'])

if __name__ == '__main__':
    demo()

__all__ = ["ContextIndex",
           "ConcordanceIndex",
           "TokenSearcher",
           "Text",
           "TextCollection"]
```

[MODULES](#) | [INDEX](#)

© Copyright 2017, NLTK Project. Last updated on May 21, 2017. Created using [Sphinx](#) :